

Como a solução funciona, do comando à saída

Versão final · branch **lucas** · 30/09/2026 · métrica oficial (máximo 1,10)

A solução lê peças jurídicas, encontra cada citação de jurisprudência ou de legislação e diz se ela é **real** (existe na base oficial, com o identificador), **inventada** (completa, mas não existe) ou **incompleta** (faltam dados para identificar um único registro). Um único comando faz tudo; a classe é sempre decidida por uma consulta determinística à base, e dois modelos de linguagem abertos ajudam a *ler* o que o texto ruim esconde.

1,0999

no dev oficial (1,09992 no leaderboard do Kaggle)

+0,035

sobre só regras na medida limpa (rodada 3, inédita): 1,0278 → 1,0627

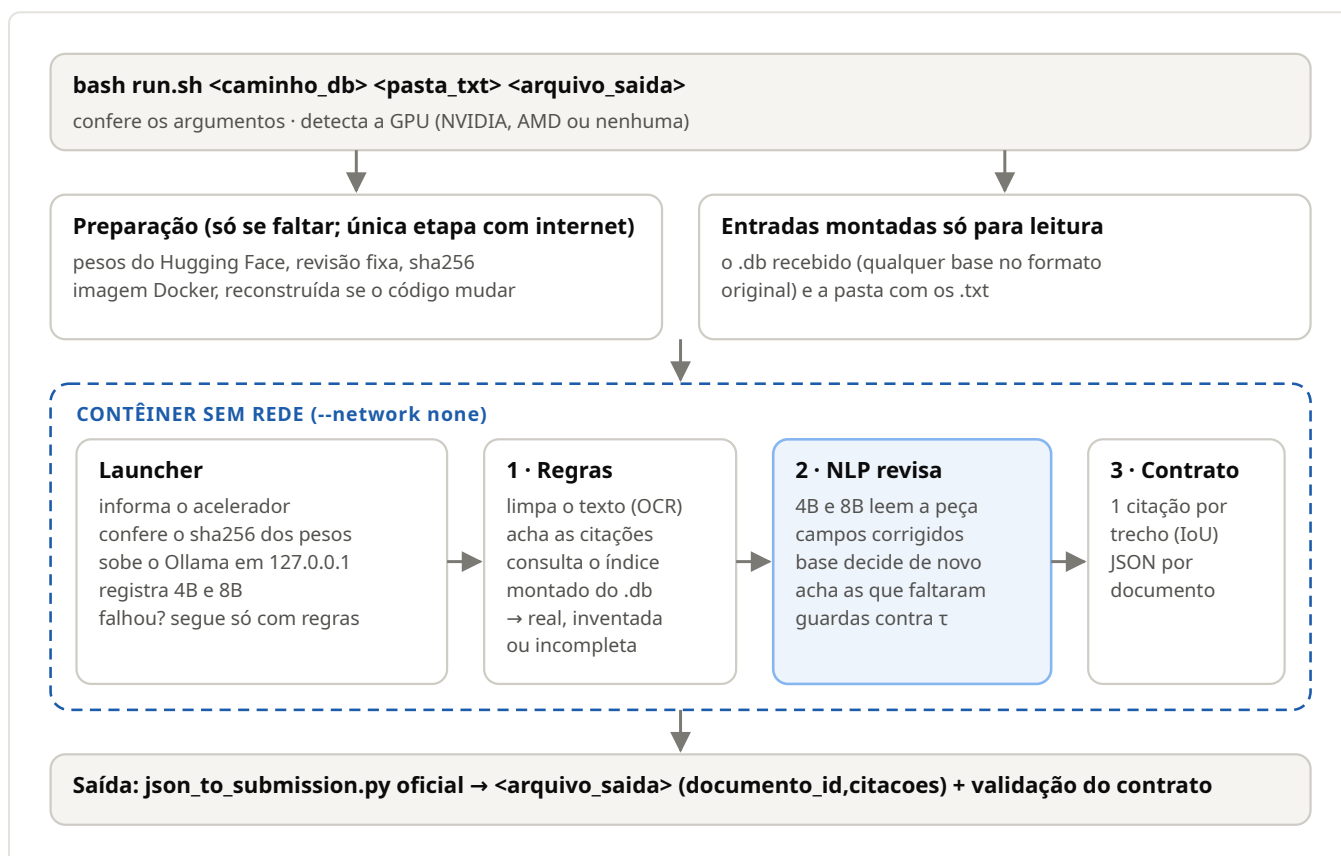
$\tau = 0$

em 39 conjuntos: nenhuma citação inventada saiu como real

2 GPUs

a mesma imagem rodou de ponta a ponta em NVIDIA e em AMD

Visão geral



1 · O comando e a preparação

```
bash run.sh <caminho_db> <pasta_txt> <arquivo_saida>
# ex.: bash run.sh data/desafio1_bracis.db data/txt saida/submission.csv
```

- 1 Confere os argumentos (base existe, pasta tem `.txt`) e detecta a GPU: `nvidia-smi` → NVIDIA; `/dev/kfd` → AMD; nenhuma → só regras. Pode ser forçada com `GPU=nvidia|amd|cpu`.
- 2 **Pesos.** Se faltarem em `modelos/`, baixa os dois declarados em `model_manifest.json` (Hugging Face, revisão fixa) e confere o sha256. Principal: `unsloth/Qwen3-4B-Instruct-2507-GGUF`; conjunto: `Qwen/Qwen3-8B-GGUF` (Q4_K_M, Apache-2.0; 7,5 GB juntos).

- 3 **Imagem.** Constrói a imagem Docker do fabricante encontrado (bases fixadas por digest; Ollama 0.33.2). A imagem leva o hash do código: sem mudança, não reconstrói nem usa a rede. Base e pesos **nunca** entram na imagem.
- 4 **Execução sem rede** (`--network none`): o `.db` e os textos entram só para leitura; os pesos, de `modelos/`.
- 5 **Saída.** Um JSON por documento vira `<arquivo_saida>` pelo `json_to_submission.py` oficial; o `validar_saida` confere o contrato (spans, classes, ids que existem na base).

2 • Dentro do contêiner: o launcher

- informa o acelerador que o Ollama encontrou (*cuda*, *rocm* ou CPU);
- confere o sha256 dos dois pesos contra o manifesto;
- sobe o servidor Ollama só em `127.0.0.1`, com `OLLAMA_NO_CLOUD=1` (sem acesso a *ollama.com*), um pedido por vez e os dois modelos carregados juntos (~12 GB de VRAM);
- registra os dois modelos e roda o pipeline; **qualquer falha** (peso ausente, hash divergente, servidor que não sobe) faz a execução seguir só com as regras, sem parar.

3 • O pipeline de cada documento

Etapa 1 • Regras (determinísticas)

- Limpeza com mapa de posições:** corrige OCR ($O \leftrightarrow 0$, $I \leftrightarrow 1$, $S \leftrightarrow 5$, $rn \leftrightarrow m$), junta palavras e números quebrados, sem perder a posição original de cada caractere — o span da saída aponta para o texto verdadeiro.
- Extração:** um catálogo de padrões (classe + número, CNJ, súmula, artigo + diploma, tema, julgado descrito por tribunal, ano e relator) e âncoras gerais que crescem a partir de pistas (“art.”, “Súmula”, uma classe processual) para formas novas. Distratores do cabeçalho (número dos próprios autos, OAB, protocolo, fls., valor da causa) ficam de fora.
- Resolução:** no início, a base recebida vira um índice pelo *cabeçalho* de cada registro (o número próprio do acórdão, a súmula, o artigo) — separa o processo de quem só o cita. A citação vira uma chave canônica e a consulta dá a classe: um candidato → **real** (com o id do Jusbrasil); nenhum → **inventada**; vários ou faltando dados → **incompleta**.

Etapa 2 • Camada de NLP (revisão pelos dois modelos)

A peça é cortada em trechos de até 3.000 caracteres; cada modelo lista as citações de cada trecho num JSON restrito por schema (temperatura 0, semente 42): o *trecho* copiado como está e os campos corrigidos (tribunal, classe, número, UF, diploma, artigo...). O trecho é localizado no texto por comparação tolerante a OCR — o que o modelo inventar e não estiver no texto é descartado.

SITUAÇÃO	O QUE A CAMADA FAZ
coincide com uma citação das regras	renormaliza: monta a forma canônica com os campos lidos e consulta a base de novo (ex.: número ilegível por OCR → a base encontra o registro → real)
o span das regras é muito maior que a citação	recorta para o trecho do modelo e resolve de novo pelas regras
citação que as regras não viram	acrescenta , se passar pelas guardas, e a classe sai da mesma consulta à base

Etapa 3 • Garantia de contrato

No máximo uma citação por trecho ($\text{IoU} < 0,5$ entre as citações do documento; sem isso, a métrica recusaria a submissão inteira) e a saída no formato do contrato.

O modelo nunca decide a classe. Ele só lê campos; quem decide é sempre a mesma consulta à base. E uma citação que as regras já resolveram como real não é renormalizada pelo modelo.

4 • Um exemplo, passo a passo

Caso real: documento gen_n1_013 do conjunto de estresse de OCR pesado (dev com 32% dos caracteres corrompidos).

texto: ... Impoe-se a observancia do art. 1º, I, «'q', da Lei Complementar n.64/1990 , ...
(→ tabulação · · caractere invisível U+2060 no meio de "Lei" · · quebra de linha)

regras: acham a citação, mas não conseguem ler a lei (o caractere invisível parte "Lei")
→ sem a lei, não há consulta → incompleta

4B: {"tipo": "artigo", "diploma": "LC64/1990", "artigo": "1"}

8B: {"tipo": "artigo", "diploma": "LC64/1990", "artigo": "1"}

guardas: a lei tem pista no próprio trecho ("Complementar n.64/1990")? sim ·
os dois modelos levam ao mesmo registro? sim

base: art. 1º da LC 64/1990 → 1 registro → real (id 11304039), igual ao gabarito

Exemplo de guarda (ilustrativo):

texto: ... nos termos da Resolução nº 188 do CNJ ...

modelo: {"tipo": "processo", "numero": "188"}

guarda: "Resolução" antes do número bloqueia → não vira citação (seria um falso positivo)

5 • Proteções e caminhos de reserva

SE ACONTECER...	A SOLUÇÃO...
sem GPU, pesos ausentes, sha256 divergente, servidor que não sobe	segue só com as regras (1,0999 no dev)
o 8B não carrega	segue só com o 4B
a execução com GPU falha	o <code>run.sh</code> repete só com as regras e ainda entrega o arquivo
média de tempo acima de 40 s por documento, ou 3 h no total	desliga a camada e o resto sai só com as regras; o 1º documento (que carrega os modelos na GPU) fica fora da média — correção de 29/09, depois que uma partida a frio de 101 s desligou a camada
o modelo lê um número que não está no trecho	recusa (dígitos só do próprio trecho, grupos inteiros)
tribunal, classe ou lei sem pista no texto	recusa
os dois modelos levam a registros diferentes	fica a resposta das regras
duas citações no mesmo trecho	fica uma só (maior confiança, depois a mais longa)

6 • Dois modelos: vale a pena?

O que o 8B acrescenta ao 4B

- **parecidos com o dev** (10 conjuntos, o formato prometido para o teste final): +0,001 na média; acima em 6, igual em 4, **abaixo em 0**
- **textos difíceis** (OCR pesado/extremo, redação de outros LLMs, 14 conjuntos): +0,004 na média; até +0,022
- **todos os 39**: +0,002; acima em 16, igual em 22, abaixo em 1 (uma citação, registrada no protocolo)
- $\tau = 0$ com um ou com dois modelos

O que custa

- tempo: ~9 → ~18 s por documento (e ~5 min a mais na partida)
- VRAM: ~4 → ~12 GB (limite: 24 GB)
- +5 GB de pesos para baixar
- um modelo a mais para carregar — coberto: sem o 8B, segue só com o 4B

Decisão: manter os dois. No formato do teste final, o segundo modelo nunca piorou e soma um pouco justamente nas formas de citação inéditas com ruído; nos textos ruins, soma mais. O custo cabe folgado nas regras (que não limitam tempo) e há volta automática para um modelo só. `CACA_ENSEMBLE=0` desliga o 8B, se for preciso.

7 · Resultados por tipo de texto

TIPO DE TEXTO	CONJUNTOS	SÓ REGRAS	NLP 4B	NLP 4B+8B
Formato oficial, como o dev	6	1,0987	1,0996	1,0996
Formas de citação inéditas	6	1,0766	1,0910	1,0927
Armadilhas e texto poluído	6	1,0938	1,0948	1,0951
Peças escritas por outros LLMs	6	0,9936	1,0361	1,0381
OCR leve (8% dos caracteres)	3	1,0897	1,0974	1,0974
OCR moderado (16%)	4	1,0596	1,0772	1,0819
OCR pesado (32%)	4	1,0045	1,0618	1,0641
OCR extremo (49%)	4	0,9465	1,0283	1,0368
Todos	39	1,0484	1,0741	1,0763

Rodadas 2 e 3: conjuntos gerados depois de congelar o código e medidos uma única vez, ao vivo. Demais: código final com as respostas dos modelos guardadas (diferença medida entre ao vivo e guardado: até 0,004). $\tau = 0$ em todas as células. Detalhes em `docs/benchmarks/` e em `tools/PROTOCOLO_NLP.md`.

8 · Requisitos, reprodução e verificação

Requisitos

- Docker (NVIDIA: com o NVIDIA Container Toolkit; AMD: `/dev/kfd` e `/dev/dri`)
- `python3` (só biblioteca padrão)
- GPU com ~12 GB livres; ~30 GB de disco (pesos, imagem e importação dos pesos)
- internet só na preparação; a execução é offline

Tempos medidos (dev, 26 documentos)

- ~18 s por documento com os dois modelos (GPU de 16 GB)
- partida: ~5–10 min (sha256 e importação dos pesos), uma vez por execução
- só regras: ~0,1 s por documento

```
# o ponto de entrada, como a organização roda
bash run.sh data/desafiol_bracis.db data/txt /tmp/saida.csv
# verificação completa: build, nada na imagem, sem rede, contrato, nota, fallback
bash tools/verificar_submissao.sh
# testes (100)
python3 -m unittest discover -s tests
```

Código, dados de referência e protocolo de avaliação: repositório da equipe (branch lucas). Números: `docs/benchmarks/` (medições ao vivo das rodadas 2 e 3 e tabela do código final). Detalhes de método e limitações: `tools/PROTOCOLO_NLP.md` e `README.md`.